VQL LSP Server — Usage Guide

How to get VQL validation in front of an AI agent, so the agent *knows* it can validate a query before submitting it to Velociraptor.

The problem

The VQL language server (`velociraptor lsp`) is built and working. It exposes two ways to validate VQL:

1. File-based (LSP protocol) — a client opens a `.vql` document and the server pushes diagnostics, or the client pulls them (`textDocument/diagnostic`). This is what opencode uses: configure the server in `opencode.json`, open a `.vql` file, and the agent sees diagnostics in its tool results.
2. Fileless (CLI) — `velociraptor lsp --check "<query>"` validates a query string with no document, no editor, no file:

```bash
$ velociraptor lsp --check "SELECT * FROM Artifact.Windows.Sys.Users()" # exit 0, no output (clean)

$ velociraptor lsp --check "SELECT * FROM Artifact.Windows.Sys.Users(foo=1)"
line 1 col 42: Unknown argument 'foo' for artifact 'Artifact.Windows.Sys.Users'
```

The fileless path is the natural fit for the agent workflow (draft query → validate → submit via the Velociraptor API). But an agent only knows a capability exists if it is discoverable. Right now `--check` is invisible: nothing in the agent's context advertises it.

How the LSP gets its artifact registry (hybrid mode)

The `lsp` command runs in hybrid mode: like the `gui`/`collector` commands it accepts an optional `--datastore` flag (defaults to a temp datastore), starts a local gRPC API server, connects to it as the superuser, and fetches the artifact registry over the API. Consequences worth knowing:

- Custom artifacts work. Any artifact stored in the datastore (uploaded via the GUI, or placed at `<datastore>/artifact_definitions/...`) is visible to validation. A bare `velociraptor lsp` with no config works too — it generates a fresh local config in a temp datastore and validates against the built-in artifacts.
- Local-only. Validation is a local process — no external Velociraptor server or authentication is needed. The local API server binds loopback only and the superuser connection uses the generated frontend certificate.
- Port collisions are handled automatically. The command tries the default API port (8001) and, if it is already taken (e.g. a production server on the same machine as the MCP+LSP), silently uses a free port instead. No flags to set.
- Deployment options. If the agent/MCP/LSP must run on a different machine from Velociraptor, copy custom artifacts into a local datastore (e.g. `velociraptor lsp --datastore /path/to/datastore`) or run the LSP on the server box where it shares the real datastore.

This document catalogs the ways to make VQL validation discoverable to agents. Every option below is real — implemented, tested, and verified in this repository — with working examples you can copy. The full source of each artifact (skill markdown, tool typescript, MCP configs) is included so you can implement the option that works best in your environment.

Why an agent can't just "find" the CLI

Agents do not discover tools by exploring. They call what they are given:

- Skills — loaded on demand when the agent's task matches a description.
- Custom tools / plugins — appear in the agent's function list.
- MCP tools — appear in the agent's tool list via the MCP protocol.

Everything else requires the agent to guess the command exists, run `--help`, and parse the output. Explicit wiring beats hoping.

Option 0 — Baseline: `velociraptor lsp --check` (no agent integration)

The raw capability. Works from any shell, script, or CI job. It is the building block every other option wraps.

```
# Syntax / unknown plugin / unknown function / unknown argument / artifact param
velociraptor lsp --check "SELECT * FROM pslist(foo=1)"
velociraptor lsp --check "SELECT * FROM Artifact.Windows.Sys.Users(badparam=1)"

# Point at a datastore containing custom artifacts (optional)
velociraptor lsp --datastore /path/to/datastore --check "SELECT * FROM Artifact.Custom
```

Without `--datastore`, the command uses (or generates) a config in a temp datastore and validates against the built-in artifacts — zero setup. With `--datastore`, custom artifacts in that datastore are validated too.

- Pros: zero infrastructure, scriptable, precise line/col output, no external network or auth (local loopback API only).
- Cons: invisible to agents unless wrapped; first run generates a local config + starts a local API server (a few seconds).
- Status: ▨ implemented and tested.

Option 1 — Agent skill (works today, zero infrastructure)

A skill is markdown + optional scripts that the agent loads when its task matches the skill description. It is the fastest way to make `--check` discoverable in opencode and similar agent frameworks.

Implementation (this repo)

Skill file at `~/.config/opencode/skills/velociraptor-vql-validation/SKILL.md` documenting the `--check` command, the diagnostic format, and examples. When an agent is asked to write or check VQL, the skill loads and tells it to use `velociraptor lsp --check` before submitting queries.

The skill frontmatter `description` is what makes it discoverable — it auto-loads whenever the agent's task mentions writing, checking, or debugging VQL. The body includes:

- Quick start: `velociraptor lsp --check "<query>"` (clean → exit 0, no output; errors → one `line N col N: message` per problem).
- A worked example with `Artifact.Windows.Sys.Users(foo=1)` → `line 1 col 42`.

- Guidance on when to use it (before API submission, when writing YAML artifact query fields, while debugging).
- A validation-coverage table (syntax, plugins, functions, keyword args, artifact existence, artifact parameters).
- Notes: the binary must be on PATH; the first run loads the artifact repository and takes a few seconds; output positions are 1-based; multi-statement documents are supported.

Copy the SKILL.md into any opencode install to make validation discoverable there too.

The full skill file (~/.config/opencode/skills/velociraptor-vql-validation/SKILL.md):

````
---
name: velociraptor-vql-validation
description: Validate VQL queries against the real Velociraptor plugin and artifact re
---

# Velociraptor VQL Validation

Validate any VQL query locally before it goes near the server. The validator
runs against the same plugin, function, and artifact registry the server
would use, so what it catches is what the server would reject.

## Quick start

```bash
velociraptor lsp --check "<query>"
```

- **Clean query** → exit 0, no output.
- **Errors** → one line per problem, `line N col N: message`, exit 0.

Example:

```bash
$ velociraptor lsp --check "SELECT * FROM Artifact.Windows.Sys.Users(foo=1)"
line 1 col 42: Unknown argument 'foo' for artifact 'Artifact.Windows.Sys.Users'
```

## When to use

Run this whenever you are about to:

1. Submit a query via the Velociraptor API (collect, hunt, notebook).
2. Insert a query into a YAML artifact as a `query` field.
3. Debug why a query failed — the validator catches typos in plugin and
   function names, wrong keyword arguments, and syntax errors with positions.

## What it validates

| Class | Example |
|---|---|
| Syntax errors | `SELECT * FROM` → `unexpected token "<EOF>"` |
| Unknown plugins | `SELECT * FROM bogusplugin()` |
| Unknown functions | `SELECT bogusfunc() FROM pslist()` |
| Unknown keyword arguments | `SELECT * FROM pslist(foo=1)` |
| Artifact existence | `SELECT * FROM Artifact.Bogus.Nope()` |
| Artifact parameters | `Artifact.Windows.Sys.Users(badparam=1)` |
````

```
## Notes

- The binary must be on PATH: `velociraptor` (built from the repo or
  installed to `~/.local/bin/velociraptor`).
- First run loads the full artifact repository and may take a few seconds.
  Subsequent runs in the same process are fast.
- Line and column are 1-based.
- Multi-statement documents are supported; a syntax error in one statement
  does not hide errors in the others.

## Related

- `VQL-LSP-DESIGN.md` — design of the language server.
- `VQL-LSP-USAGE.md` — all the ways to expose validation to agents.
- `VQL-LSP-TESTS.md` — test coverage and verified examples.
```

- Pros: instant, no build, no config, matches an established pattern (the repo already ships several `velociraptor-*` skills).
- Cons: prose, not a structured call — the agent must follow the instructions; weaker than a tool for guaranteeing invocation.
- Status: ▨ implemented and verified. Skill file created at `~/.config/opencode/skills/velociraptor-vql-validation/SKILL.md` (see below).

Option 2 — Custom tool / plugin (structured call in the agent's function list)

A custom tool wraps `--check` (or the validation engine directly) and surfaces as a first-class callable: input a query string, output structured diagnostics. The agent sees it without loading anything.

Implementation (this repo — opencode tool)

Tool file at `~/.config/opencode/tools/validate-vql.ts`, following the same convention as the repo's other custom tools (default export `tool()`):

```
import { tool } from "@opencode-ai/plugin"
import { execFile } from "node:child_process"
import { promisify } from "node:util"

const execFileAsync = promisify(execFile)

export default tool({
  description:
    "Validate a VQL query against the real Velociraptor plugin and artifact registry.
  args: {
    query: tool.schema
      .string()
      .describe("The VQL query to validate, e.g. SELECT * FROM pslist()"),
  },
  async execute(args) {
    try {
      const { stdout } = await execFileAsync("velociraptor", [
        "lsp",
        "--check",
        args.query,
      ])
```

```
      if (stdout.trim() === "") {
        return JSON.stringify({ valid: true, diagnostics: [] }, null, 2)
      }

      const diagnostics = stdout
        .split("\n")
        .filter((line) => line.trim() !== "")
        .map((line) => {
          // e.g. "line 1 col 42: Unknown argument 'foo' ..."
          const match = line.match(/^line (\d+) col (\d+): (.*)$/)
          if (match) {
            return {
              line: parseInt(match[1], 10),
              column: parseInt(match[2], 10),
              message: match[3],
            }
          }
          return { line: 0, column: 0, message: line }
        })

      return JSON.stringify(
        { valid: diagnostics.length === 0, diagnostics },
        null,
        2,
      )
    } catch (err) {
      // velociraptor not on PATH or failed to start.
      const message = err instanceof Error ? err.message : String(err)
      return JSON.stringify({
        valid: false,
        error:
          "Failed to run velociraptor lsp --check. Is the binary on PATH? " +
          message,
      })
    }
  },
})
```

The tool shells out to `velociraptor lsp --check` and parses the output into structured `{line, column, message}` diagnostics, returning `{valid, diagnostics}`. It is auto-loaded from the opencode tools directory — no `opencode.json` change needed.

- Pros: guaranteed discoverability, structured JSON output, no MCP dependency.
- Cons: opencode-specific (or framework-specific); does not help other clients.
- Status: ▨ implemented and verified. The tool appears in the opencode agent's function list after restart and was exercised from inside the live agent: bad query → `valid: false` with 3 diagnostics (positions 1:15, 1:25, 1:69); clean queries → `valid: true`. Also bundles cleanly with `bun build`.

Option 3 — MCP server (framework-neutral, any MCP client)

Expose VQL validation as an MCP tool so any MCP-capable agent (opencode, Claude Code, Cursor, etc.) can call it. Two sub-options:

Option 3a — mcpls: wrap the LSP server as MCP

mcpls is a universal LSP→MCP bridge. It starts our LSP server (velociraptor lsp) and exposes its capabilities (get_diagnostics, get_hover, get_completions, …) as MCP tools.

Configure ~/.config/mcpls/mcpls.toml:

```
[workspace]
# mcpls restricts file access to these roots; list the repo(s) you validate.
roots = ["/path/to/your/repo"]

# vql is not among mcpls' built-in language mappings, so add one.
# Note: language_extensions is nested UNDER [workspace].
[[workspace.language_extensions]]
extensions = ["vql"]
language_id = "vql"

[[lsp_servers]]
language_id = "vql"
command = "velociraptor"
args = ["lsp"]
file_patterns = ["**/*.vql"]
# The LSP server needs a few seconds to load the full plugin/artifact
# registry on first start.
timeout_seconds = 60
# Restrict the bridge to the diagnostics routes only (optional).
handles = ["diagnostics"]
```

Then add mcpls to your MCP client config and the VQL tools become available. The get_diagnostics tool takes a single file_path argument (absolute path); mcpls opens that file as a virtual LSP document and returns the diagnostics as a JSON tool result. The first call after server start may return "LSP server 'vql' is still initializing" — retry after a few seconds.

Verified example (MCP call against lsp-test/bridge-test.vql):


```
{"diagnostics":[{"range":{"start":{"line":1,"character":42},"end":{"line":1,"character
  "severity":"error",
  "message":"Unknown argument 'foo' for artifact 'Artifact.Windows.Sys.Users'"},
 {"range":{"start":{"line":2,"character":22},"end":{"line":2,"character":30}},
  "severity":"error",
  "message":"Unknown argument 'badarg' for plugin 'pslist'"}]}
```


- Pros: instant reuse of the LSP's full protocol surface; works for any language server, not just VQL; framework-neutral.
- Cons: an extra process (mcpls) in the middle; tool semantics are generic LSP calls (diagnostics still need a document path to open).
- Status: ▨ implemented and verified end-to-end. The prebuilt binary on this system was glibc 2.39 (system has 2.35), so it was rebuilt from source with the local Rust toolchain (v0.3.8). Config template above was completed with roots = ["<your repo>"] (mcpls restricts file access to workspace roots) and worked as-is.

Option 3b — native validate_vql tool in mcp-velociraptor

The existing Go MCP server at ~/projects/mcp-velociraptor/ talks to the Velociraptor gRPC API. A validate_vql tool fits naturally as one more tool there, sharing the same codebase.

- Pros: one MCP server for everything (list clients, run artifacts, and validate VQL);

no extra bridge process; can validate against the exact plugin/artifact registry of the connected instance.

- Cons: mcp-velociraptor requires a valid Velociraptor API config to start (it is fundamentally an API client). Validation itself is local, so the tool works even when the connected instance is unreachable.
- Status: ▨ implemented and verified end-to-end against a live Velociraptor instance (see the verified examples below).

Implementation (mcp-velociraptor validate_vql)

internal/tools/validate_vql.go, following the existing list_clients.go pattern:

```
var ValidateVQLTool = mcp.NewTool("validate_vql",
    mcp.WithDescription("Validate a VQL query and return diagnostics "+
        "(syntax errors, unknown plugins/functions, unknown arguments, "+
        "artifact parameter checks)."),
    mcp.WithString("query",
        mcp.Description("The VQL query to validate"),
    ),
)

func HandleValidateVQL(ctx context.Context, request mcp.CallToolRequest) (*mcp.CallToo
    query := mcp.ParseString(request, "query", "")
    // Either shell out to `velociraptor lsp --check` or import the
    // validation engine from www.velocidex.com/golang/velociraptor/vql/lsp.
    ...
}
```

Register in cmd/mcp-velociraptor/main.go:

```
srv.AddTool(tools.ValidateVQLTool, tools.HandleValidateVQL)
```

The validate_vql handler builds a lazy, cached validation registry from the built-in plugins and the full artifact repository (via lsp.BuildRegistryWithArtifacts — the in-process sibling of the hybrid LSP server's BuildRegistryFromAPIClient), then runs the query through it. The registry is built once per process and reused, so repeated calls are fast after the first.

Note: importing the vql/lsp package requires the same replace directives as the Velociraptor module (participle/v2, the local vfilter checkout). Also note the vfilter replace must be added to mcp-velociraptor's own go.mod: replace directives do NOT propagate from a dependency module.

- Pros: single MCP server for Velociraptor work; agent gets a clean validate_vql(query) call.
- Cons: needs the mcp-velociraptor build to track the LSP engine.

The mcp-velociraptor server requires a valid Velociraptor API config to start (see its README for VELOCIRAPTOR_API_CONFIG / --config). An API connection is mandatory at startup — this is an API client by design — but validate_vql itself is purely local: it builds its own registry from the built-in plugins and artifacts and does not need the connected instance.

Verified examples (MCP call tools/call validate_vql, run against a live Velociraptor API on localhost:8001):

```
{"query":"SELECT upcase(str='x'), bogusfunc() FROM Artifact.Windows.Sys.Users(foo=1)",
 "valid":false,
```

```
 "diagnostics":[
   {"line":1,"column":15,"severity":"error",
    "message":"Unknown argument 'str' for function 'upcase'"},
   {"line":1,"column":25,"severity":"error",
    "message":"Unknown function 'bogusfunc'"},
   {"line":1,"column":69,"severity":"error",
    "message":"Unknown argument 'foo' for artifact 'Artifact.Windows.Sys.Users'"}]}
```

A valid query (`SELECT * FROM Artifact.Windows.Sys.Users()`) returns `{"valid":true, "diagnostics":[]}`.

The same server, connected to the live instance, also answered `tools/call list_clients` with the real registered clients (client IDs, hostnames, OS, online status), confirming the shared registry and API connection work together.

Option 4 — Editor client (future): VS Code / Neovim / Zed

The LSP server speaks the standard Language Server Protocol, so it is client-agnostic. Everything implemented so far — diagnostics, hover, completion, and document symbols — will work in any editor that speaks LSP: VS Code, Neovim, Emacs, Zed, and others.

This is NOT implemented yet — it is documented here so the path is clear for anyone who wants it.

What it would look like

A thin client extension (on the order of ~100 lines) that:

1. Tells the editor to spawn `velociraptor lsp` for `.vql` files (the language server runs over stdio, exactly as opencode uses it).
2. Wires the editor's LSP client to that process.

After that, the editor gets the usual IDE experience for VQL for free:

- Red squiggles under syntax errors, unknown plugins/functions/artifacts, and unknown keyword arguments (with exact line/column).
- Hover documentation for functions, plugins, arguments and their types.
- Autocomplete for plugins, functions, artifacts, LET variables, and argument names.
- A document outline of the query structure (LETs, queries, columns).

Caveat: UTF-16 vs byte offsets

Our position mapping is byte-based, which is correct for the ASCII VQL that is overwhelmingly typical. VS Code (and some other editors) use UTF-16 character offsets internally, so a string literal containing non-ASCII characters (emoji, non-Latin text) could show diagnostics a character or two off until a position-conversion shim is added. This is a small, well-understood fix, and irrelevant for ordinary VQL.

Not needed for the AI-agent workflow

The original motivation for this server was giving AI agents a pre-flight validation loop (draft → validate → fix → submit via API). Editors are a natural byproduct of speaking standard LSP, not a requirement of that workflow — which is why this option is documented as future work rather than built today.

Decision guide

Need	Pick
Validate a query from a shell/CI script	Option 0 (`--check`)
Make it discoverable in opencode today, minimal work	Option 1 (skill)
Structured, guaranteed tool call in one agent framework	Option 2 (custom tool)
Framework-neutral MCP, reuse full LSP surface	Option 3a (mcpls)
One MCP server for all Velociraptor work incl. validation	Option 3b (mcp-velociraptor `validate_vql`)
Full IDE experience for humans (VS Code, Neovim, Zed)	Option 4 (editor client — future)

Implementation status

Option	Status
0 — `lsp --check`	✅ implemented, tested (see VQL-LSP-TESTS.md)
1 — skill	✅ implemented and verified (`velociraptor-vql-validation` skill)
2 — custom tool	✅ implemented and verified (`validate-vql` opencode tool)
3a — mcpls	✅ rebuilt from source, config complete, verified end-to-end
3b — mcp-velociraptor `validate_vql`	✅ implemented and verified end-to-end against a live instance
4 — editor client (VS Code etc.)	☐ future option — standard LSP means the server already speaks it; a ~100-line client extension wires it up (see Option 4)